

```

import random

class BlacklistManager:
    def __init__(self, all_channels):
        self.all_channels = all_channels
        self.active_sequence = list(all_channels)
        self.blacklist = []
        self.stats = {ch: {"success": 0, "total": 0} for ch in all_channels}
        self.threshold = 0.6 # 60% de sucesso mínimo exigido

    def get_pdr(self, channel):
        """Calcula o Packet Delivery Ratio (Taxa de Entrega)"""
        s = self.stats[channel]
        return s["success"] / s["total"] if s["total"] > 0 else 1.0

    def record_attempt(self, channel, success):
        """Registra o resultado de uma transmissão"""
        self.stats[channel]["total"] += 1
        if success:
            self.stats[channel]["success"] += 1

        # Avalia se deve ir para a blacklist
        pdr = self.get_pdr(channel)
        if pdr < self.threshold and channel not in self.blacklist:
            print(f"⚠️ ALERTA: Canal {channel} detectado como
instável (PDR: {pdr:.2%}). Blacklisting...")
            self.update_blacklist(channel, add=True)
        elif pdr >= self.threshold and channel in self.blacklist:
            print(f"✅ RECUPERADO: Canal {channel} voltou a ficar
estável. Reintegrando...")
            self.update_blacklist(channel, add=False)

    def update_blacklist(self, channel, add=True):
        """Atualiza a lista de canais ativos"""
        if add and channel in self.active_sequence:
            self.active_sequence.remove(channel)
            self.blacklist.append(channel)
        elif not add and channel in self.blacklist:
            self.blacklist.remove(channel)
            self.active_sequence.append(channel)
            self.active_sequence.sort()

    def get_next_hop(self, asn):
        """Calcula o canal ignorando os que estão na Blacklist"""
        if not self.active_sequence:
            return None # Todos os canais estão ruins!

        # A sequência de saltos agora só usa canais 'limpos'
        idx = asn % len(self.active_sequence)
        return self.active_sequence[idx]

# --- Simulação de Ambiente ---
# Canais 11 a 26 (Padrão 802.15.4)
manager = BlacklistManager(list(range(11, 27)))

print(f"Canais Ativos Iniciais: {manager.active_sequence}\n")

# Simulação de 50 transmissões
for asn in range(50):
    ch = manager.get_next_hop(asn)

    # Simulando interferência pesada nos canais 15 e 16 (ex: Wi-Fi próximo)
    if ch in [15, 16]:
        success = random.random() > 0.8 # 80% de chance de falha
    else:
        success = random.random() > 0.1 # 10% de chance de falha

    manager.record_attempt(ch, success)

print(f"\n--- Estado Final da Rede ---")
print(f"Canais Ativos: {manager.active_sequence}")
print(f"Blacklist: {manager.blacklist}")

```

(A)

```

import random

class PeriodicBlacklistManager:
    def __init__(self, channels, analysis_period=20):
        self.all_channels = channels
        self.active_channels = list(channels)
        self.blacklist = []
        self.analysis_period = analysis_period # Período de slots para
reanálise

        # Estatísticas da janela atual
        self.window_stats = {ch: {"success": 0, "total": 0} for ch in
channels}

        # Limiares (Thresholds)
        self.min_pdr = 0.7 # Bloqueia se PDR < 70%
        self.recover_pdr = 0.85 # Desbloqueia apenas se teste na blacklist
for > 85%

    def record_event(self, channel, success):
        """Registra o sucesso ou falha no slot atual"""
        self.window_stats[channel]["total"] += 1
        if success:
            self.window_stats[channel]["success"] += 1

    def run_reanalysis(self):
        """
        Função de Auditoria: Executada a cada 'analysis_period'.
        Decide quem entra e quem sai da lista negra.
        """
        print("\n--- 🔍 Iniciando Reanálise Periódica da Rede ---")

        new_blacklist = []
        new_active = []

        for ch in self.all_channels:
            stats = self.window_stats[ch]

            # Se não houve tráfego no canal, mantemos o estado atual
            if stats["total"] == 0:
                if ch in self.active_channels: new_active.append(ch)
                else: new_blacklist.append(ch)
                continue

            pdr = stats["success"] / stats["total"]

            # Lógica de Decisão
            if ch in self.active_channels:
                if pdr < self.min_pdr:
                    print(f"❌ BLOQUEADO: Canal {ch} (PDR: {pdr:.1%}) -> Movido
para Blacklist")
                    new_blacklist.append(ch)
                else:
                    new_active.append(ch)
            else:
                # Reanálise de canais que estavam bloqueados (Probing)
                if pdr >= self.recover_pdr:
                    print(f"✅ DESBLOQUEADO: Canal {ch} (PDR: {pdr:.1%}) ->
Retornando à operação")
                    new_active.append(ch)
                else:
                    print(f"⏸️ MANTIDO: Canal {ch} (PDR: {pdr:.1%}) -> Segue na
Blacklist")
                    new_blacklist.append(ch)

            self.active_channels = new_active
            self.blacklist = new_blacklist

        # Reseta estatísticas para a próxima janela
        self.window_stats = {ch: {"success": 0, "total": 0} for ch in
self.all_channels}
        print("--- ✅ Reanálise Concluída ---\n")

    def get_next_channel(self, asn):
        """Retorna o canal para o slot atual.
        Se o canal agendado estiver na blacklist, escolhe o próximo ativo
disponível."""
        target_ch = self.all_channels[asn % len(self.all_channels)]

        if target_ch in self.active_channels:
            return target_ch, "Normal"
        else:
            # Probing: De vez em quando, mesmo bloqueado, o nó tenta usar o
canal

            # para colher estatísticas de reanálise.
            if random.random() < 0.2: # 20% de chance de teste
                return target_ch, "Probing (Teste)"

            # Se não for teste, usa um canal reserva da lista ativa
            alt_ch = self.active_channels[asn % len(self.active_channels)]
            return alt_ch, "Substituto"

# --- Simulação de Execução ---
channels = [11, 12, 13, 14, 15]
manager = PeriodicBlacklistManager(channels, analysis_period=15)

for asn in range(45):
    ch, mode = manager.get_next_channel(asn)

    # Simulando interferência pesada no canal 13
    if ch == 13:
        success = random.random() > 0.6 # 60% de falha
    else:
        success = random.random() > 0.1 # 10% de falha

    manager.record_event(ch, success)

    # Verifica se é hora da reanálise programada
    if (asn + 1) % manager.analysis_period == 0:
        manager.run_reanalysis()

```

(B)